IFAC Discrete-Event System Design
Gandia Beach, Spain, 2009

Ladder diagram implementation of Control Interpreted Petri Nets: a state equation approach

Marcos V. Moreira* Daniel S. Botelho João C. Basilio***

* *COPPE - Electrical Engineering Program, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil (e-mails: moreira@dee.ufrj.br, basilio@dee.ufrj.br).*
** *Electrical Engineering Department, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil (e-mail: daniel.botelho@poli.ufrj.br)*

Abstract: Control Interpreted Petri Nets (CIPN) provide high level structured controllers for discrete event systems. However, discrete event control systems are usually implemented in PLCs using a graphical symbolic language called Ladder diagram, which is a low level programming language not suitable for the analysis and design of complex discrete event control systems. Recent works address the problem of converting control systems designed using CIPNs into Ladder diagrams. However, these works either lead to difficult implementation schemes or do not consider important practical aspects related to the implementation of the control code on a PLC. In this paper, a simple conversion method that establishes simple transformation rules from the CIPNs to Ladder diagrams is proposed. The method is based on the state equation of the Petri net; thus modifications on the discrete event control system modeled by a CIPN can be easily taken into account in order to modify an existing Ladder diagram developed in accordance with the methodology proposed here.

Keywords: Petri net, ladder diagram, programmable logic controller.

1. INTRODUCTION

In the modeling and analysis of discrete event dynamic systems, one of the tools frequently used is the so-called Petri net (Peterson, 1981; Murata, 1989; Cassandras and Lafortune, 2008). Using Petri nets, it is possible to model and visualize behaviors such as concurrency, parallelism, synchronization and resource sharing (David and Alla, 2005). However, for the control of a discrete event dynamic system, the discrete event controller is timed and synchronized with external events. In order to deal with these requirements, Control Interpreted Petri Nets (CIPN) have been proposed (David and Alla, 1995; Uzam and Jones, 1998; Uzam et al., 1996; Jimenez et al., 2001). These Petri nets are extensions of ordinary Petri nets with particular structures to deal with actuators and sensors.

However, the control of discrete event systems is usually implemented using Programmable Logic Controllers (PLCs), which can be programmed using four different languages defined in the standard IEC 1131-3 (IEC 1131-3, 1993; Ohman et al., 1998): (*i*) ladder diagram; (*ii*) function block diagram; (*iii*) structured text; (*iv*) instruction list. Among these four available languages, Ladder is the most adopted one; it is a low level programming language consisting of graphic symbols representing, for instance, contacts and coils, being therefore not suitable for analysis and design of complex discrete event control systems.

There are several works in the literature that address the problem of converting complex control codes designed using CIPNs into Ladder diagrams for implementation (Uzam and Jones, 1998; Uzam et al., 1996; Jimenez et al., 2001; Lee et al., 2004). In Fabian and Hellgren (1998) and Hellgren et al. (2005), two problems related to the implementation of control codes designed using automata and SFCs (Sequential Function Charts) are introduced: (*i*) the need for a choice between conflicting transitions, and (*ii*) the avalanche effect. Moreover, methods to overcome these problems are also proposed. However, some aspects of the conversion are not clear and a general procedure for the conversion has not been obtained. In Uzam and Jones (1998) a CIPN, called the Automation Petri Net, is proposed. This CIPN includes actions and sensor readings as formal structures in the Petri net model and also introduces enabling and inhibitor arcs, which can enable and/or disable transitions through the use of leading-edge, falling-edge and level of markings. The conversion of Automation Petri Nets into Ladder diagrams is explained by using the so-called Token Passing Logic (TPL) method, which uses the evolution of the tokens through the Petri net as the main mechanism for controlling the flow of the control logic. Although the method proposed in Uzam and Jones (1998) was successfully implemented on a discrete manufacturing system, some implementation problems such as the avalanche effect have not been explicitly considered.

In this paper, a new method for the conversion of a CIPN into a Ladder diagram is proposed. The method is based on the state equation of the Petri net and uses the structure of the Petri net, described by its incidence matrices, to obtain conditions for the firing of transitions

and to describe the flow of the control logic. The Ladder diagram is organized in such a way to avoid the avalanche effect. From the input incidence matrix, it is possible to find all transitions that may have an effective conflict, allowing the programmer to choose a priority order for these transitions. The structure of the Ladder program is simple and allows that modifications on the discrete event control system, modeled by the CIPN, be easily implemented in the existing Ladder diagram.

This paper is organized as follows. In Section 2, Petri nets are defined and the state equation is presented. In Section 3, the definition of the Control Interpreted Petri Nets used in the paper is presented. In Section 4, some implementation aspects related to the use of PLCs are discussed. Section 5 presents the conversion method and an illustrative example. Finally, conclusions are drawn in Section 6.

2. PETRI NETS BACKGROUND

2.1 Petri nets and the state equation

A Petri net graph is a bipartite graph that contains two types of nodes: places and transitions. The places are represented by circles and the transitions by bars, and these two types of nodes are connected through arcs. The formal definition of a Petri net graph is given as follows.

Definition 1. A Petri net graph is a weighted bipartite graph (P, T, A, w), where P is the set of finite places, T is the set of finite transitions, $A \subseteq (P \times T) \bigcup (T \times P)$ is the set of arcs, and $w : (P \times T) \bigcup (T \times P) \rightarrow \mathbb{N}$ is the weight function, with $w(p_i, t_j) = 0$ and $w(t_j, p_i) = 0$ if and only if $(p_i, t_j) \notin A$ and $(t_j, p_i) \notin A$, respectively. □

The set of places is represented in this paper as $P = \{p_1, p_2, \ldots, p_n\}$ and the set of transitions as $T = \{t_1, t_2, \ldots, t_m\}$. Thus, $|P| = n$ and $|T| = m$, where $|.|$ denotes cardinality.

In a Petri net, events are associated with transitions and places are associated with the conditions that must be met in order for these events to occur. The mechanism that indicates if these conditions are met is obtained through assignment of tokens to the places. The number of tokens assigned to a place p_i is represented by $x(p_i)$, where $x : P \rightarrow \mathbb{N}$. Thus, a marking of a Petri net is the row vector $\underline{x} = [x(p_1)\, x(p_2) \ldots x(p_n)]$ formed by the number of tokens in each place p_i, for $i = 1, \ldots, n$. The state of a Petri net is defined by the marking $\underline{x}$, and a marked Petri net is defined as the five-tuple $(P, T, A, w, \underline{x})$, where (P, T, A, w) is, according to definition 1, the Petri net graph and $\underline{x}$ is a marking of the set of places P. A marked Petri net in which all places, for all reachable markings, can have at most one token is called a safe Petri net and the places are called safe.

A transition t_j is said to be enabled when the number of tokens in each one of its input places is greater than or equal to the weight of the arcs connecting the places to transition t_j, *i.e.*, t_j is enabled if and only if

$$x(p_i) \geq w(p_i, t_j), \text{ for all } p_i \in I(t_j), \qquad (1)$$

where $I(t_j)$ denotes the set of input places of t_j. If transition t_j is enabled for a marking $\underline{x}$ and the event associated with t_j occurs, transition t_j fires and a new marking $\bar{\underline{x}}$ is achieved. The evolution of the markings is given by the following equation:

$$\bar{\underline{x}}(p_i) = x(p_i) - w(p_i, t_j) + w(t_j, p_i), i = 1, \ldots, n. \qquad (2)$$

It is easy to verify that the evolution of the state vector given by (2), after a firing of transition t_j, can be described by the following state equation:

$$\bar{\underline{x}} = \underline{x} + \underline{u}W, \qquad (3)$$

where $\underline{u}$ is a row vector formed by zeros with the exception of the *j-th* element that is equal to 1 to represent the firing of transition t_j, and W is the $m \times n$ incidence matrix given by:

$$W = W_{out} - W_{in}, \qquad (4)$$

where $W_{in} = [w_{ij}^{in}]$, with $w_{ij}^{in} = w(p_i, t_j)$, is the input incidence matrix and $W_{out} = [w_{ij}^{out}]$, with $w_{ij}^{out} = w(t_j, p_i)$, is the output incidence matrix.

Notice that, using (3), one can obtain all states of the Petri net, reachable from a state $\underline{x}$, if $\underline{u}$ is a feasible firing vector, *i.e.*, if the transition associated with $\underline{u}$ is enabled. Moreover, it is easy to verify that a transition t_j is enabled for a marking $\underline{x}$ if and only if all elements of vector $\underline{x} - \underline{u}W_{in}$, where $\underline{u}$ is the firing vector associated with t_j, are non-negative.

2.2 Conflicts

Let T_K be the set of output transitions of a place $p_i \in P$. Then, a structural conflict, denoted by $K =< p_i, T_K >$, occurs when the cardinality of the set T_K is greater than one, *i.e.*, the Petri net graph (P, T, A, w) has a structural conflict if and only if there exists k and $l \in \mathbb{N}^*$, such that $w_{ik}^{in} \geq w_{il}^{in} > 0$ for some $p_i \in P$. Therefore, the existence of a structural conflict can be verified by inspection of the matrix W_{in}.

It is important to notice that the structural conflict is independent from the marking $\underline{x}$ of the Petri net. An effective conflict, on the other hand, depends on its marking (David and Alla, 2005).

Definition 2. An effective conflict, denoted by $K^E =< p_i, T_K, \underline{x} >$ is the existence of a structural conflict $K =< p_i, T_K >$, and of a marking $\underline{x}$, such that the transitions in the set T_K are enabled by $\underline{x}$ and the number of tokens in p_i is less than the sum of all the weights of the arcs from p_i to the transitions of the set T_K. □

Definition 2 shows that an effective conflict is not a structural property of the Petri net and, therefore, cannot be identified from the incidence matrices of the Petri net.

The transitions of T_K that are involved in an effective conflict are said to be conflicting transitions, while transitions that can fire in any order or simultaneously are called concurrent transitions. Thus, it is possible to have a structural conflict and not an effective conflict if the number of tokens in the input place p_i is sufficient to fire all transitions involved in the structural conflict, for all reachable markings of the Petri net.

3. CONTROL INTERPRETED PETRI NET

The CIPN defined in this paper has a formal structure to deal with sensors and actuators and, therefore, it is

synchronized with external events. Moreover, in order to include timers, the CIPN is also T-timed, where T can be partitioned into $T = T_0 \bigcup T_D$, where T_0 is the set of transitions with no firing delays and T_D is the set of transitions that have firing delays, *i.e.*, T_D is the set of timed transitions.

The CIPN has a condition part, that is associated with the transitions and an operation part that is associated with the places. The condition part includes events, such as the reading of sensors, and conditions, that must be satisfied to fire the transitions. The operation part includes actions that can be of two types: level actions or impulse actions. Thus, in the CIPN considered in this paper the inputs are associated with transitions and the outputs are associated with places.

Definition 3. A Control Interpreted Petri Net is a six-tuple (N, C, E, O, O_I, D), where $N = (P, T, A, w, \underline{x})$ is the marked Petri net with initial state $\underline{x}$, $C = \{C_1, \ldots, C_m\}$ and $E = \{E_1, \ldots, E_m\}$ are the sets of conditions and input events, associated with the m transitions in T, respectively, O and O_I are the sets of level actions and impulse actions, associated with safe places in P, respectively, and $D = \{d_j : t_j \in T_D\}$ is the set of firing delays associated with the timed transitions of T_D. □

Notice that in definition 3 it is supposed that every transition has a condition associated with it. If condition C_j associated with t_j is not explicitly specified, then $C_j = 1$, *i.e.*, the logical condition C_j is true. Moreover, if the input event E_j is not explicitly specified, then $E_j = e$, the always occurring event (David and Alla, 2005). Therefore, according to definition 3, all transitions of T have boolean expressions associated with them. These expressions can be written as $R_j = C_j \cdot E_j$, for $j = 1, \ldots, m$, where R_j is called the receptivity of transition t_j.

The CIPN must be deterministic in the sense that if there is an effective conflict (definition 2) between two or more transitions, then the receptivities associated with them must be mutually exclusive, defining priorities for the transitions involved in the effective conflict.

Remark 1. Some CIPNs proposed in the literature use three different types of arcs: ordinary arcs; enabling arcs; and inhibitor arcs. Ordinary arcs are defined from places to transitions or from transitions to places. However, inhibitor arcs and enabling arcs are only defined from places to transitions. The enabling arc has the same features of a self-loop between the place and the transition, *i.e.*, it enables the transition when the number of tokens in the input place is greater than the weight of the enabling arc, but the input place does not lose tokens when the transition fires. The inhibitor arcs, on the other hand, have a different meaning. When the number of tokens in the input place is greater than the weight of the inhibitor arc, the transition is not enabled. Although inhibitor arcs are not defined for the Petri net controller described in this paper, the procedure for the conversion of the Petri net controller into a Ladder diagram can also be used in these cases with a small modification in the input incidence matrix of the Petri net. This is explained in section 5. □

4. PROGRAMMABLE LOGIC CONTROLLERS

A PLC operates executing scan cycles, that consist of three main steps: (*i*) read and store the inputs; (*ii*) execute the entire user program code; and (*iii*) write the outputs. Therefore, the PLC reads signals generated from sensors of the plant, uses these values as the inputs of the program code, executes the program code, and, in the end, generates signals that are applied to the plant. In general, input events are associated with the rising edge or the falling edge of signals that are generated by sensors.

In the theory of supervisory control, events are not supposed to occur simultaneously. However, in practical applications, when the supervisor is implemented on a PLC two or more events may be seen by the controller as simultaneous events due to the scan cycle. In order to understand this, notice that a rising edge of a signal is detected when the level of the signal is low at a scan cycle and, in the next scan cycle, the level is high. Since a scan cycle lasts a period of time, two or more events may occur in one scan cycle. In Fabian and Hellgren (1998), two problems related to the implementation of a supervisor in Ladder language are discussed: (*i*) the avalanche effect; and (*ii*) the need for a decision when there are conflicting transitions.

The avalanche effect occurs when two or more sequential transitions have the same expression for the receptivities associated with them and the implementation of the Ladder diagram transposes more than one transition when the receptivities become true. In order to overcome this problem, Fabian and Hellgren (1998) suggest that the rungs of the Ladder program could be rearranged such that only the first transition is transposed when the receptivities become true. However, this procedure is not easily implemented if there are loops in the Petri net. Moreover, a general procedure to deal with the avalanche effect is not obtained.

In the case of conflicting transitions, the programmer must choose a priority order for the transitions to guarantee that the CIPN is deterministic. In Fabian and Hellgren (1998), it is proposed a method that arranges the rungs in the Ladder program in an order such that the last rung is associated with the transition that has the highest priority. Although this method is well defined, it would be easier to establish a connection between the CIPN and its Ladder implementation, if the conflicting transitions have mutually exclusive receptivities as in a Grafcet (David, 1995). This is the procedure adopted in this paper.

The PLC can be programmed in four languages defined in the standard IEC 1131-3 (IEC 1131-3, 1993), being the Ladder diagram the most used language. The Ladder diagram is a symbolic language that uses contacts, coils, timers, counters, comparison instructions and also simple math instructions. In the next section examples of Ladder diagrams are presented to illustrate the proposed conversion method.

5. CONVERSION OF CIPN INTO LADDER DIAGRAMS

The method proposed in this paper is based on the state equation given by (3). The method is simple and can deal with the implementation problems described in section 4.

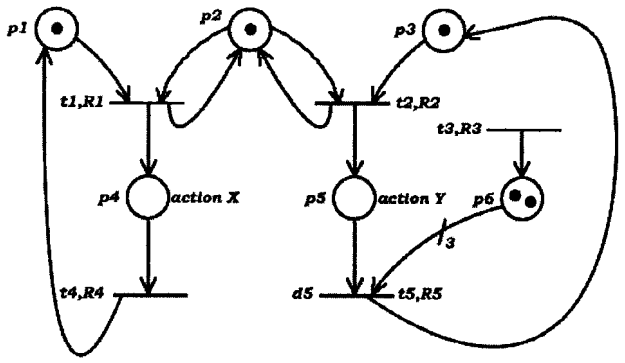

Fig. 1. Control Interpreted Petri Net.

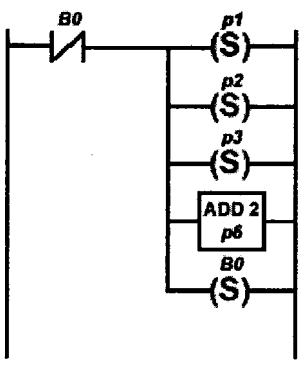

Fig. 2. Initialization module.

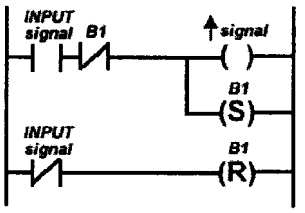

Fig. 3. Ladder diagram representation of an event associated with the rising edge of a signal.

The method consists in dividing the Ladder code in five modules. The first module is associated with the initialization of the Petri net, *i.e.*, it defines the initial marking. The second module is associated with the occurrence of events, and the third module with the conditions for the firing of the transitions, which are related to the receptivities of the transitions and to the input incidence matrix. The fourth module describes the evolution of the tokens in the Petri net using its incidence matrix, and the fifth module defines which actions will be set.

This conversion technique provides a complete visualization of the CIPN in the Ladder code, allowing the CIPN to be easily extracted from the Ladder diagram, and modifications in the CIPN to be easily implemented.

In the following subsections each one of the five modules for the conversion is described and an example of a CIPN, shown in figure 1, is used to illustrate the conversion.

5.1 Initialization module

The first rung of the Ladder diagram describes the initialization of the Petri net. This rung contains a normal closed (NC) contact associated with an internal binary variable that, in the first scan cycle, logically energizes coils associated with safe places that have an initial token, and ADD blocks associated with places that can have more than one token. The ADD block must set the number of tokens that these places have in the initial marking. After the first scan cycle the NC contact is opened.

In figure 2 the first rung of the Ladder diagram for the CIPN of figure 1 is shown. The internal binary variable associated with the NC contact is $B0$. Notice that, since the initial marking of the CIPN is $\underline{x} = [1\,1\,1\,0\,0\,2]$, and p_1, p_2 and p_3 are safe places, then coils are associated with p_1, p_2 and p_3, and an ADD block with the value of $x(p_6) = 2$ is associated with p_6.

5.2 Module of the events

Events are, in general, associated with the rising edge or the falling edge of sensor signals. In order to be able to detect the transition of the signal level, some PLCs have special contacts that keep the logic true for only one scan cycle when a transition in the signal level occurs. In this paper, it is proposed a simple way of detecting the transition level of the input signal using NC and NO contacts, and SET and RESET coils. In figure 3 it is shown the Ladder representation of an event associated with the rising edge of a signal, where $B1$ is an internal binary variable. The falling edge representation can be obtained replacing in figure 3 the SET (RESET) coil with a RESET (SET) coil, the NC contacts with NO contacts, and the NO contact with a NC contact.

5.3 Module of conditions for the firing of transitions

This module describes the enabling conditions presented in the input incidence matrix W_{in} and the receptivities. This module has m rungs, where each rung corresponds to the conditions for the firing of a transition.

The basic idea is to describe the firing rule of a transition, namely that, a transition t_j is fireable when the number of tokens in each of its input places is greater than the weight of the arcs connecting the places to t_j and condition C_j associated with receptivity R_j is true, and transition t_j fires when event E_j associated with R_j occurs.

The conditions associated with the input places can be obtained by inspection from W_{in}, that contains the weights of the arcs connecting the input places to the transitions. For a transition t_j, these conditions in the Ladder diagram are expressed by the series connection of normal open (NO) contacts or comparison instructions with the inequalities $p_i \geq w_{ij}^{in}$, for all $i = 1, 2, \ldots, n$, such that $w_{ij}^{in} > 0$. A NO contact can be used if the place is safe, otherwise a comparison instruction must be used. When the inequality in a comparison instruction is true its contact is closed, and if the inequality is not true its contact is opened.

To represent the firing of transition t_j, the boolean expression for the receptivity R_j (implemented, in general, with a simple association of NO and NC contacts) is connected in series with the conditions of the input places of t_j. It is important to remark that if an effective conflict is possible, then the receptivities must be mutually exclusive

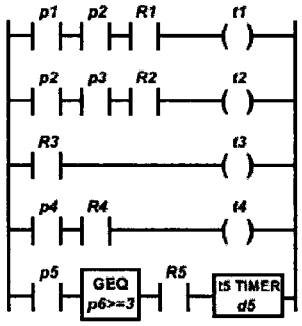

Fig. 4. Module of conditions for the firing of transitions.

and they must define a priority order among the conflicting transitions to have a deterministic controller. This priority order is chosen by the programmer and can be achieved by defining appropriately the conditions associated with the receptivities.

If a transition is timed, then a timer with the preset value equal to the delay associated with the timed transition must be used. In these cases, the time is also a firing condition for the transition. After the delay time, the timer energizes a coil indicating that the delay time has elapsed. In this paper it is considered that the timer resets its accumulated value if its corresponding rung is opened.

In figure 4 it is shown the module of conditions for the firing of transitions of the Petri net of figure 1, obtained from the input incidence matrix W_{in} given by:

$$W_{in} = \begin{array}{c} \begin{matrix} p_1 & p_2 & p_3 & p_4 & p_5 & p_6 \end{matrix} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 3 \end{bmatrix} \end{array} \begin{matrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ t_5 \end{matrix} . \quad (5)$$

Notice that, as it can be easily checked from the second column of W_{in}, there is a structural conflict between transitions t_1 and t_2. From the Petri net marking it can be verified that this conflict can be an effective conflict and, therefore, receptivities R_1 and R_2 must be mutually exclusive. Supposing that transition t_1 has higher priority than transition t_2, the priority order can be achieved defining:

$$R_1 = E_1.C_1, \text{ and } R_2 = E_2.C_2 = E_2.C_{t_2}.\left(\bar{E}_1 + \bar{C}_1 + \bar{p}_1\right),$$

where $\bar{E}_1$, $\bar{C}_1$ and $\bar{p}_1$ denote the nonoccurrence of event E_1, condition C_1 is not satisfied and absence of a token in place p_1, respectively, and C_{t_2} denotes other conditions associated only with transition t_2. Therefore, transition t_2 will be receptive to event E_2 if and only if t_2 is enabled, condition C_{t_2} is true and the event associated with t_1, E_1, does not occur, or transition t_1 is not enabled, or condition C_1 is false. It is important to remark that if $E_1 = e$, the always occurring event, then condition $\bar{E}_1$ in receptivity R_2 could be dropped since t_1 fires as soon as it is enabled if condition C_1 is true.

Notice also that transition t_5 is timed, and thus, a timer is inserted in the rung of t_5.

5.4 Module of the Petri net dynamics

After the firing of a transition t_j, the number of tokens in the Petri net must be updated. This process is described by the state equation of the Petri net. Therefore, for the construction of the module that represents the dynamics of the Petri net, it is used its incidence matrix W. The module has m rungs, where each rung is associated with a transition and expresses the changes in the place markings after the firing of the transition.

If an action associated with a place is an impulse action, then it must be done in only one scan cycle. A possible way of doing this is to keep the token in the place for just one scan cycle (Uzam et al., 1996). However, in this paper, it is considered the case that the impulse action is enabled for just one scan cycle even if the token remains in the place. A new occurrence of the action will happen only if a different token is deposited in the corresponding place. In order to do so, for the transitions that lead to a place with an impulse action, an internal binary variable must be set to zero when the transition fires. This internal binary variable is a control variable for the impulse action, that guarantees that the action will occur during only one scan cycle. This control variable is used in the module of the actions.

In figure 5 it is shown the Ladder code obtained from the incidence matrix W:

$$W = \begin{array}{c} \begin{matrix} p_1 & p_2 & p_3 & p_4 & p_5 & p_6 \end{matrix} \\ \begin{bmatrix} -1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & -3 \end{bmatrix} \end{array} \begin{matrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ t_5 \end{matrix} . \quad (6)$$

In this example, action X is supposed to be a level action and action Y is supposed to be an impulse action. Notice that in the rung associated with transition t_2 there is the introduction of a coil with the binary variable $B5$. This binary variable is used in the module of the actions to control the impulse action Y.

5.5 Module of the actions

The module of the actions associates an output coil to the places that have an impulse or level action. In the case of impulse actions, it is also included a SET coil for the internal binary variable that controls the impulse action.

In figure 6 it is shown the module of actions for the example of figure 1. Notice that the impulse action Y is controlled by the binary variable $B5$, *i.e.*, when the contact of p_5 is closed, the SET coil of B_5 is energized and, in the next scan cycle, the NC contact of B_5 is opened, guaranteeing that action Y occurs in only one scan cycle.

Some important observations can be done with respect to the method described in this section.

Remark 2.

- The arrangement of the rungs proposed in this paper avoids the avalanche effect since each marking of the CIPN remains unchanged for at least one scan cycle in its Ladder implementation.

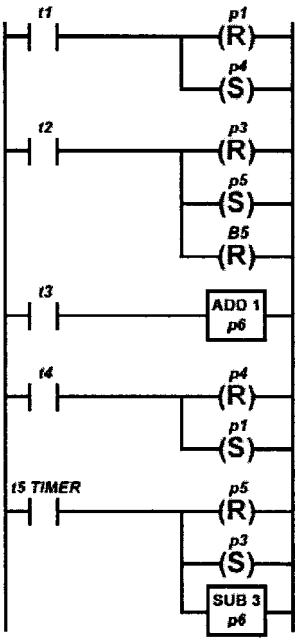

Fig. 5. Module of the Petri net dynamics.

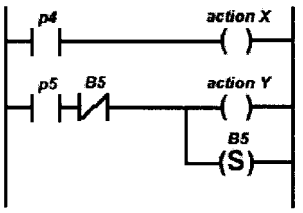

Fig. 6. Module of the actions.

- Assuming that there are l distinct events different from e associated with the rising edge or the falling edge of external signals, then the maximum number of rungs in the Ladder program obtained from the method is $(2m + 2l + n + 1)$. Although the number of rungs could be smaller, the Ladder diagram proposed in this work shows clearly the Petri net structure, since W_{in} and W can be obtained from the Ladder code. Morevover, the receptivities and actions can be easily related to their corresponding transitions and places, respectively. Therefore, the CIPN can be directly obtained from the Ladder diagram and vice-versa.
- If the CIPN has inhibitor arcs, then a modification must be done in the input incidence matrix in order to use the method. In this case, this modified input incidence matrix will not be used in the computation of the incidence matrix given by (4), since the firing of the transition with an inhibitor arc does not change the number of tokens in the input place. However, it can be used for the obtention of the conditions in the module of conditions for the firing of transitions. The procedure includes the weight of the inhibitor arc, in the corresponding entry of W_{in}. This value will be translated to the Ladder code as the inequality $p_i < w_{ij}^{in}$ in the module of conditions for the firing of transitions. This simple modification allows the use of inhibitor arcs in the method of conversion proposed in this paper.

6. CONCLUSIONS

In this paper, a simple method for the conversion of discrete event control systems described by CIPN into Ladder diagrams for implementation on a PLC is proposed. The main feature of this method is that it allows a direct obtention of the CIPN from the Ladder code and vice-versa. Therefore, modifications in the CIPN can be easily implemented in the Ladder code. Another important contribution of the method is that the avalanche effect is naturally avoided by the Ladder implementation of the CIPN.

REFERENCES

Cassandras, C.G. and Lafortune, S. (2008). *Introduction to Discrete Event Dynamic Systems.* Springer, New York.

David, R. (1995). Grafcet: a powerful tool for specification of logic controllers. *IEEE Transactions on Control Systems Technology*, 3, 253–268.

David, R. and Alla, H. (1995). Petri nets for modeling of dynamic systems - a survey. *Automatica*, 30, 175–202.

David, R. and Alla, H. (2005). *Discrete, Continuous and Hybrid Petri Nets.* Springer.

Fabian, M. and Hellgren, A. (1998). PLC-based implementation of supervisory control for discrete event systems. In *Proceedings of the 37th IEEE Conference on Decision and Control*, 3305–3310.

Hellgren, A., Fabian, M., and Lennartson, B. (2005). On the execution of sequential function charts. *Control Engineering Practice*, 13, 1283–1293.

Jimenez, I., lopez, E., and Ramirez, A. (2001). Synthesis of Ladder diagrams from Petri nets controller models. In *Proceedings of the 2001 IEEE International Symposium on Intelligent Control*, 225–230.

Lee, G.B., Zandong, H., and lee, J.S. (2004). Automatic generation of Ladder diagram with control Petri net. *Journal of intelligent Manufacturing*, 15, 245–252.

IEC 1131-3 (1993). *Programmable controllers - Part 3: Programming Languages.* International Electrotechnical Commission.

Murata, T. (1989). Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77, 541–580.

Ohman, M., Johansson, S., and Arzen, K. (1998). Implementation aspects of the PLC standard IEC 1131-3. *Control Engineering Practice*, 6, 547–555.

Peterson, J.L. (1981). *Petri net theory and the modeling of systems.* Prentice Hall, Upper Saddle River, NJ.

Uzam, M. and Jones, A.H. (1998). Discrete event control system design using automation Petri nets and their Ladder diagram implementation. *Int J Adv Manuf Technol*, 14, 716–728.

Uzam, M., Jones, A.H., and Ajlouni, N. (1996). Conversion of Petri nets controllers for manufacturing systems into Ladder logic diagrams. In *IEEE Conference on Emerging Technologies and Factory Automation*, 649–655.